

KARNATAK LAW SOCIETY'S
GOGTE INSTITUTE OF TECHNOLOGY

UDYAMBAG, BELAGAVI-590008

(An Autonomous Institution under Visvesvaraya Technological University, Belagavi)
(APPROVED BY AICTE, NEW DELHI)

Department of Electronics and Communication



Project Report On

Cloud-Based Autonomous Tracking of Battery Health for Safe EV Operation

Submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

In

Electronics and Communication Engineering

Submitted by

Pranali. Patil -USN: 2GI21EC081

Rutuja. Halgekar -USN: 2GI21EC103

Shrinidhi. Pattar -USN: 2GI21EC134

Sneha. Murgod -USN: 2GI21EC143

Under the Guidance of

Dr. Saurav Mitra

(Professor)

2024 – 2025

KARNATAK LAW SOCIETY'S
GOGTE INSTITUTE OF TECHNOLOGY
UDYAMBAG, BELAGAVI-590008
(An Autonomous Institution under Visvesvaraya Technological University, Belagavi)
(APPROVED BY AICTE, NEW DELHI)

Department of Electronics and Communication



CERTIFICATE

Certified that the project entitled **“Cloud-Based Autonomous Tracking of Battery Health for Safe EV Operation”** carried out by

Ms.Pranali Patil
Ms.Rutuja Halgekar
Ms.Shrinidhi Pattar
Ms.Sneha Murgod

USN: 2GI21EC081,
USN: 2GI21EC103,
USN: 2GI21EC134,
USN: 2GI21EC143,

Students of KLS Gogte Institute of Technology, Belagavi, can be considered as a bonafide work for partial fulfillment for the award of **Bachelor of Engineering in Electronics and Communication** of the Visvesvaraya Technological University, Belagavi during the year 2024- 2025.

It is certified that all corrections/suggestions indicated have been incorporated in the report. The project report has been approved as it satisfies the academic requirements prescribed for the said Degree.

Guide
(Dr. Saurav Mitra)

HOD
(Dr Supriya S. Shanbhag)

Principal
(Dr M.S.Patil)

Date:

Final Viva-Voce

	Name of the examiners	Date of Viva -voce	Signature
1.			
2.			

DECLARATION BY THE STUDENTS

We, ***Pranali Patil, Rutuja Halgekar, Shrinidhi Pattar, Sneha Murgod***, hereby declare that the project report entitled “**Cloud-Based Autonomous Tracking of Battery Health for Safe EV Operation**” to KLS Gogte Institute of Technology, Belagavi, in partial fulfillment of the Degree of **Bachelor of Engineering in Electronics and Communication**, is a record of the project carried out at ***KLS Gogte Institute of Technology Campus***. This report is for academic purposes.

We further declare that the report has not been submitted and will not be submitted, either in part or full, to any other institution or University for the award of any diploma or degree.

Name of the student	USN	Signature
Pranali Patil	2GI21EC081	
Rutuja Halgekar	2GI21EC103	
Shrinidhi Pattar	2GI21EC134	
Sneha Murgod	2GI21EC143	

Place: Belagavi

Date:

ACKNOWLEDGEMENT

Any achievement does not depend solely on individual efforts but on the guidance, encouragement, and cooperation of intellectuals, elders, and friends. Some personalities, in their capacities, have helped us in carrying out this project work. We would like to take this opportunity to thank them all.

First and foremost, we would like to thank **Dr. Saurav. Mitra**, our project guide. We greatly thank him for his guidance and cooperation throughout the journey.

We want to convey our heartfelt thanks to projecting coordinators, **Prof. Vaidehi Deshpande**, Head of the Department, **Dr. Supriya S. Shanbhag**, and the Principal, **Dr. M.S.Patil** for allowing us to work on this topic.

We thank all the faculty members and Technical/Administrative Staff of the Department of Electronics and Communication Engineering for their constant support and encouragement.

We would also like to thank everyone who has helped us in this endeavor.

Last but not least, we would like to thank our parents and friends, who gave us valuable suggestions to improve our project.

Pranali. Patil
Rutuja. Halgekar
Shrinidhi. Pattar
Sneha. Murgod

Table of Contents

		Content	Page No.
	I.	Declaration	i
	ii.	Acknowledgment	ii
	iv.	Table of contents	iii
	v.	List of Tables	iv
	vi.	List of Figures	v
	vii	List of Abbreviations	vi
1.		Introduction, Objective, and Methodology	
	1.1	Introduction	8
	1.2	Objectives	9-10
	1.3	Methodology	10-15
2.		Chapter II	
	2.1	Literature Survey	16-18
3.		Chapter III	
	3.1	Problem definition	19
	3.2	Initial Design	19-27
4		Chapter IV	
	4.1	Experimentation	28
	4.2	System calibration	28-35
	4.3	Data collection for Arduino	35
	4.4	Discharge cycle of 3.7volt battery	35
	4.5	Complete setup of Arduino	36-39
	4.6	Data collection for ESP32	39
	4.7	Discharge cycle of 12-volt battery	40
	4.8	Communication between ESP32 devices	40

	4.9	Complete setup of ESP32	41-54
5		Chapter V	
	5.1	Results	55-60
	5.2	Discussion	60-61
6		Chapter VI	
	6.1	Conclusion	62
	6.2	Future Scope	62-64
	6.3	Case study	64-67
7		References	68
Appendix-A		Source code	

List of Tables

Table No.	Title (Caption)	Page No.
1	Voltage sensor calibration	30
2	Current sensor calibration	32
3	DHT11 sensor calibration	33

List of Figures

Figure No.	Title (Caption)	Page No.
1	PCB for current & voltage sensor module	21
2	Voltage sensor calibration	29
3	Current sensor calibration	31
4	DHT11 sensor calibration	33
5	Final setup- output of Arduino connection	36
6	Final setup of Electric vehicle BMS	41
7	Final setup of RSU	42
8	$V > V_{min}$ so SOC is 0%	55
9	$V_{min} \geq V$ so SOC is 43%	55
10	$V \geq V_{min}$ so SOC is 75%	56
11	The output is shown on a serial monitor in Arduino Ide	56
12	Measured parameters value	57

13	Delivery success	58
14	Delivery fails	58
15	$V < V_{min}$	59
16	$V < V_{min}$ so message sent	59
17	BMS status @ vehicle	60
18	BMS status @ RSU	60

List of Abbreviations

Abbreviation	Description
SOC	State of Charge

CHAPTER I

This Chapter deals with the introduction to the project work, its objective, and the methodology.

The introduction deals with the gist of the work carried out, objectives include the plan and specific goals to be achieved by the end of this project. The methodology includes the implementation part of this project.

1.1 INTRODUCTION:

In modern electronics, monitoring the health and status of batteries is vital for ensuring reliable operation and prolonging battery life, particularly in electric vehicles and other high-performance applications. This project provides a comprehensive solution for monitoring both a 3.7V Li-ion battery and a 12V lead-acid battery, offering a dual-purpose approach that addresses a wide range of battery types and operational requirements.

For the 3.7V Li-ion battery, the system uses an Arduino Uno to measure voltage, current, and temperature, and calculates the state of charge (SOC) based on voltage readings. The voltage sensor module provides precise voltage readings critical for SOC calculations, while the ACS712 current sensor measures the current to monitor charge and discharge cycles. The LM35 temperature sensor ensures the battery operates within safe temperature limits. These parameters are processed by the Arduino Uno and displayed in real-time on a 16x2 I2C LCD screen, offering users a clear overview of the battery's status.

The project extends its capabilities to monitor a 12V lead-acid battery, commonly used in electric vehicles. For this, the system integrates additional components, including a 25V voltage sensor module and an ACS712 current sensor for higher voltage and current ranges. A DHT11 sensor replaces the LM35 to measure both temperature and humidity, ensuring the system can track environmental conditions that affect battery performance. The monitored data is crucial for understanding the charging and discharging behavior of the battery, maintaining operational safety, and improving efficiency.

To enhance functionality, the project incorporates two ESP32 modules for wireless communication and remote monitoring. The first ESP32 acts as the sensor node, collecting parameters such as voltage, current, temperature, and humidity. It transmits this data wirelessly to the second ESP32 using the ESP-NOW protocol, a low-latency and energy-efficient communication method. The second ESP32 acknowledges data delivery and uploads the received information to the Adafruit cloud platform. This integration provides real-time data visualization on custom dashboards, enabling remote monitoring and decision-making.

By combining these features, the project offers a robust solution for monitoring and managing batteries in real-time, whether for standalone 3.7V Li-ion batteries or 12V lead-acid batteries in electric vehicles. Its scalability, reliability, and seamless integration with IoT platforms make it a valuable tool for battery health diagnostics and energy management in various applications.

1.2 OBJECTIVE:

The objective of this project is to design, implement, and evaluate a comprehensive real-time battery monitoring system for both 3.7V Li-ion batteries and 12V lead-acid batteries, addressing the needs of standalone systems and electric vehicles. The system aims to:

1. **Monitor Key Parameters:** Accurately measure and display battery voltage, current, temperature, and state of charge (SOC) using sensor modules like the ACS712 current sensor, voltage sensors, and DHT11 for environmental factors.
2. **Ensure Safety and Efficiency:** Provide insights into battery health by tracking charge/discharge cycles, SOC, and operational temperature ranges to prevent damage and enhance efficiency.
3. **Enable Wireless Communication:** Utilize ESP-NOW protocol between two ESP32 modules for low-latency, energy-efficient, and reliable data transmission.
4. **Provide Remote Monitoring:** Upload monitored parameters to the Adafruit cloud platform for real-time visualization and remote accessibility via custom dashboards.
5. **Facilitate Scalability:** Create a modular system capable of integrating additional sensors or features, such as predictive maintenance or advanced analytics, for future IoT and electric vehicle applications.

This project seeks to demonstrate a robust, scalable solution for battery health monitoring, ensuring reliability, fault detection, and operational longevity in modern electronics and electric vehicle systems.

1.3 METHODOLOGY

1.3.1 Methodology for Arduino Part of the Project

1. Component Selection:

- Arduino Uno: Chosen for its versatility and ease of use in processing sensor data.
- 3.7V Li-ion Battery: The battery to be monitored, is commonly used in various portable electronics.
- Voltage Sensor Module: Selected for accurate voltage measurement and compatibility with Arduino.
- Current Sensor (ACS712): Chosen for its ability to measure current in a range suitable for the Li-ion battery.
- Temperature Sensor (LM35): Selected for its precision in measuring temperature.
- LCD Display (I2C): Used for clear and concise display of data.
- Load (Two 1k ohm Resistors in Series): Simulates the battery load under realistic conditions.

2. Circuit Design:

- Voltage Sensor: The voltage sensor module is connected to the battery to measure the voltage. The VCC and GND pins of the voltage sensor are connected to the 5V and GND pins of the Arduino, respectively. The output pin is connected to the A0 analog input pin of the Arduino.
- Current Sensor: The ACS712 current sensor is placed in series with the load and the battery to measure the current. The VCC and GND pins of the ACS712 are connected to the 5V and GND pins of the Arduino, respectively. The output pin is connected to the A1 analog input pin of the Arduino.
- Temperature Sensor: The LM35 temperature sensor is placed close to the battery to measure its temperature. The VCC and GND pins of the LM35 are connected to the 5V and

GND pins of the Arduino, respectively. The output pin is connected to the A2 analog input pin of the Arduino.

- LCD Display: The LCD is connected using the I2C protocol. The SDA and SCL pins of the LCD are connected to the A4 and A5 pins of the Arduino, respectively. The VCC and GND pins of the LCD are connected to the 5V and GND pins of the Arduino.

3. Code Development:

- Libraries and Setup:

 - Import necessary libraries for the LCD and I2C communication.

 - Define pin assignments and constants for the sensors.

- Data Acquisition:

 - Write functions to read data from the voltage, current, and temperature sensors.

- SOC Calculation:

 - Develop a function to estimate the SOC based on the battery voltage.

- Data Display:

 - Write a function to display the voltage, SOC, current, and temperature on the LCD.

- Main Loop:

 - Continuously read sensor data, calculate SOC, and update the LCD.

4. Testing and Calibration:

- Voltage Sensor Calibration:

 - Ensure the voltage sensor provides accurate readings by comparing its output with a known reference voltage.

- Current Sensor Calibration:

 - Verify the current sensor readings using a known current source and adjust the calculations if necessary.

- Temperature Sensor Calibration:

 - Validate the temperature sensor readings against a known temperature source.

- SOC Estimation Accuracy:

 - Test the SOC calculation by comparing it with a battery's actual charge and discharge cycles.

5. Implementation:

- Hardware Setup:

Assemble the circuit as per the design, ensuring all connections are secure and correct.

- Software Upload:

Upload the developed code to the Arduino Uno using the Arduino IDE.

- Real-time Monitoring:

Power up the system and observe the readings on the LCD. Ensure that the voltage, SOC, current, and temperature are displayed correctly.

By following this methodology, the project aims to create a reliable and accurate battery monitoring system that provides real-time insights into the health and status of a 3.7V Li-ion battery. This system can be extended to other battery types and applications, making it a versatile tool for battery management.

1.3.2 Methodology for ESP32 Part of the Project:

The ESP32-based system enables real-time monitoring of key parameters such as voltage, current, temperature, and humidity from a 12V lead-acid battery. The project uses two ESP32 modules for sensing, processing, and transmitting data to the cloud for visualization and analysis. Below is the detailed methodology:

1. Hardware Setup and Connections

Sensor Node (First ESP32):

i. ACS712 Current Sensor (30A version):

- Measures the current flowing through the battery.
- Connected to the ESP32 via one of its analog input pins (e.g., A0).
- The output from the sensor is mapped to actual current values through calibration, considering the sensor's sensitivity (e.g., 66mV/A).

ii. 25V Voltage Sensor Module:

- Reads the battery's voltage by dividing the input voltage to a measurable range (0–3.3V) suitable for the ESP32's ADC pins.
- Connected to another analog input pin. Proper scaling is applied in the code to calculate the actual voltage.

- iii. DHT11 Sensor:
 - Measures temperature and humidity of the environment.
 - Connected to a digital GPIO pin of the ESP32 for communication using the DHT library.
- Receiver Node (Second ESP32):
 - Receives the transmitted data via ESP-NOW from the sensor node.
 - Connected to the same network as the Adafruit cloud for uploading the received data.
- Power Supply:
 - Both ESP32 modules are powered using a regulated 5V power supply to ensure stable operation. The sensors derive their power from the ESP32.

2. Sensor Data Acquisition

- The sensor node collects raw data from the sensors:
 - The voltage sensor outputs a scaled-down voltage value that is converted to the actual voltage of the 12V battery using a multiplier derived from the sensor's divider ratio.
 - The current sensor provides a voltage corresponding to the measured current, and this is converted to current values using calibration equations.
 - The DHT11 sensor outputs digital data directly for temperature (°C) and humidity (%).
- Calibration is critical at this stage to ensure that all sensors provide accurate measurements. Calibration includes:
 - Comparing sensor outputs against known reference values.
 - Applying offsets or scaling factors in the code to correct inaccuracies.

3. Data Processing on the Sensor Node (First ESP32)

- The ESP32 processes the sensor data using its ADC and GPIO pins.
- Voltage, current, and temperature values are converted into human-readable units and stored in variables.
- To ensure accuracy, filtering techniques (e.g., averaging) are applied to minimize noise in sensor readings.

- The sensor data is structured into a packet containing all parameters (voltage, current, temperature, and humidity).

4. Wireless Communication Using ESP-NOW Protocol

- The processed data packet is transmitted to the receiver node (second ESP32) using the ESP-NOW protocol.
 - ESP-NOW provides low-latency, peer-to-peer communication without the need for a Wi-Fi router.
 - The transmission is encrypted for security.
- After transmitting the data, the first ESP32 waits for an acknowledgment (ACK) from the receiver.
 - If the ACK is not received within a predefined time, the system logs a “Delivery Fail” message and indicates the issue to the user.

5. Data Handling and Acknowledgment by the Receiver Node (Second ESP32)

- The receiver ESP32 processes the received data packet and:
 - Confirms the integrity of the data using checksum or validation techniques.
 - Sends an acknowledgment back to the sensor node to confirm successful reception.
- If the received data is outside expected ranges (e.g., voltage exceeds 15V or temperature exceeds safe limits), the system triggers a fault alert for safety.

6. Cloud Integration with Adafruit IO

- The receiver ESP32 uses the Adafruit IO library to upload the processed data to the Adafruit cloud platform.
 - The ESP32 connects to a Wi-Fi network to establish communication with Adafruit IO.
 - Data is sent to the cloud using MQTT (Message Queuing Telemetry Transport) protocols.
- On the Adafruit cloud, real-time dashboards are created to display parameters such as:
 - Voltage (in volts).
 - Current (in amperes).
 - Temperature (in °C).
 - Humidity (in %).

7. Testing and Calibration

- i. The system undergoes rigorous testing under different conditions, such as varying loads and environmental factors.
- ii. Calibration ensures:
 - The voltage readings match the actual battery voltage under load.
 - Current readings are consistent with actual current flow measured using an ammeter.
 - The DHT11 sensor readings are cross-verified with a reliable thermometer and hygrometer.

8. Fault Detection and Alerts

The system includes fault detection mechanisms:

- If data is not received within the expected time, the transmitter logs a "Delivery Fail" message.
- If any parameter exceeds predefined thresholds, alerts are generated for user intervention.

The ESP32-based monitoring system ensures accurate data acquisition, reliable communication, and seamless cloud integration, making it a robust solution for real-time monitoring of 12V batteries in electric vehicles.

CHAPTER II

This chapter deals with the survey carried out on the existing methods based on a similar project idea.

2.1 LITERATURE SURVEY:

Title of the paper	Year of publication	Author name	Contribution	Future scope	GAP analysis
Battery Management System in Electric Vehicles	05-May-2020	A. Hariprasad	Battery Management Systems (BMS) in EVs ensure battery safety, performance, and longevity by monitoring SOC, SOH, temperature, and voltage balance.	AI for better SOC/SOH accuracy.	Improved SOC/SOH estimation is needed.
Optimal Sizing of Electric Vehicle Charging Stations Considering Urban Traffic Flow for Smart Cities	2021	Miguel.Campaña, Esteban Inga, Jorge Cárdenas	Proposed a heuristic model using Hungarian and MCFP algorithms to optimize the sizing and placement of EV charging stations based on urban traffic flow. Focused on minimizing travel time and addressing geo-referenced vehicular density.	Expand research into modeling more dynamic and heterogeneous traffic scenarios. Investigate scalability with larger datasets and additional variables like economic activity zones.	Existing studies lacked integration of microscopic vehicular traffic analysis with charging station placement optimization. Limited focus on dynamic path changes and user routing in previous works.

Lithium-Ion Battery Management System for Electric Vehicles: Constraints, Challenges, and Recommendations	27-February-2023	A. K. M. Ahasan Habib	They monitor and manage key parameters like SOC, SOH, temperature, and voltage, ensuring efficient and reliable battery operation while preventing overcharging, over-discharging, and thermal runaway.	Adaptation for different battery chemistries and sizes	Managing diverse battery chemistries and configurations.
State-of-charge and remaining charge estimation of series-connected lithium-ion batteries for cell balancing scheme	22 - September -2016	Chang Yoon Chun	Focuses on accurately estimating the state-of-charge (SOC) and remaining charge of series-connected lithium-ion batteries, crucial for effective cell balancing.	Enhanced real-time monitoring systems for improved accuracy.	High complexity in implementing advanced estimation algorithms.
Analysis Of Cell Balancing Techniques In BMS For Electric Vehicles	22-August-2022	Amar Nath	It aims to enhance battery performance, lifespan, and safety by ensuring uniform charge distribution among cells. Key techniques include passive balancing, active balancing, and hybrid methods, each offering unique advantages in managing cell voltages.	Application to larger battery packs and future battery technologies.	Ensuring safety during balancing operations to prevent thermal runaway or overcharging.
Impact of Electric Vehicle Charging on Voltage and Current	03-August-2023	Seyed Mahdi Miraftebzadeh	This research explores the impact of Electric	Harmonic Mitigation Techniques	Assessment of the environmental

Harmonics at the Point of Common Coupling			Vehicle (EV) charging on voltage and current harmonics at the Point of Common Coupling (PCC). It investigates how EV charging introduces harmonic distortions, affecting power quality in distribution networks.	Development of advanced filters and mitigation strategies to reduce harmonics introduced by EV chargers.	impact of harmonic distortions from widespread EV adoption.
---	--	--	--	--	---

CHAPTER III

3.1 Problem Definition:

The efficient and reliable operation of electric vehicles (EVs) relies heavily on continuous monitoring of critical parameters such as battery voltage, current, temperature, and environmental conditions. Traditional monitoring systems often lack real-time capabilities, seamless wireless communication, and cloud integration, making it challenging to remotely track the health and performance of EV batteries. This limitation can lead to undetected faults, reduced battery life, and potential safety hazards.

To address these challenges, there is a need to develop an IoT-based real-time monitoring system that accurately measures and transmits essential parameters wirelessly, detects faults promptly, and provides remote visualization and analysis through cloud platforms. Such a system should be scalable, energy-efficient, and reliable for modern electric vehicle applications.

3.2 Initial Design:

3.2.1. Component Selection:

Components Required for the Project:

1. Microcontrollers:

2 x ESP32 Development Boards:

- One ESP32 for the sensor node to collect and transmit data.
- Another ESP32 as the receiver node to handle incoming data and upload it to the cloud.

2. Sensors:

i. ACS712 Current Sensor (30A version):

- Measures the current flowing through the battery, providing a voltage output proportional to the current.

ii. 25V Voltage Sensor Module:

- Monitors the voltage of the 12V lead-acid battery by scaling down the input voltage to a safe range for the ESP32.

iii. DHT11 Temperature and Humidity Sensor:

- Measures the temperature and humidity in the battery's environment, ensuring it operates within safe conditions.

3. Power Supply:

i. 5V Regulated Power Supply:

- Powers the ESP32 development boards and sensors. A stable 5V supply ensures reliable operation.

ii. 12V Battery (Lead-Acid):

- Powers the system and acts as the main source of energy in the project.

4. Communication Module:

ESP-NOW Protocol (on ESP32):

- Built-in wireless communication protocol used to transmit data between the two ESP32 boards without the need for Wi-Fi routers.

5. Display:

16x2 I2C LCD Display (Optional for local display):

- Displays the real-time values of battery voltage, current, temperature, and humidity on-site.

6. Cloud Platform:

Adafruit IO Cloud Platform:

- Provides cloud-based storage and real-time visualization of the monitored data. Data from the ESP32 will be uploaded here for remote monitoring.

7. Other components :

i. Jumper Wires and Connectors:

- For making necessary connections between sensors, ESP32, and power supply.

ii. PCB:

- For prototyping the connections in the sensor node and receiver node. We have designed a PCB for sensing current, voltage, and a load of 21watts bulb.

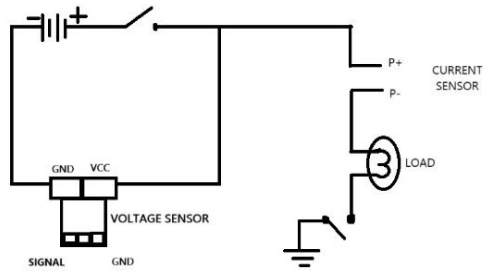


Fig.01: PCB for current & voltage sensor

Software and Libraries:

- i. **Arduino IDE:**
 - Used for programming the ESP32 boards.
- ii. **Adafruit IO Library (for cloud integration):**
 - Allows communication with Adafruit's cloud platform.
- iii. **DHT11 Library:**
 - For interfacing with the DHT11 sensor.
- iv. **ESP-NOW Library:**
 - To enable ESP32-based communication between devices.

3.2.2. Circuit Design for Arduino

Voltage Sensor:

- Connect the VCC of the voltage sensor to 5V on the Arduino.
- Connect the GND of the voltage sensor to the GND on the Arduino.
- Connect the output of the voltage sensor to the A0 analog pin on the Arduino.

Current Sensor (ACS712):

- Connect the VCC of the current sensor to 5V on the Arduino.
- Connect the GND of the current sensor to the GND on the Arduino.
- Connect the output of the current sensor to the A1 analog pin on the Arduino.
- Place the current sensor in series with the battery and the load.

Temperature Sensor (LM35):

- Connect VCC of the LM35 to 5V on the Arduino.

- Connect the GND of the LM35 to the GND on the Arduino.
- Connect the output of the LM35 to the A2 analog pin on the Arduino.

LCD Display (I2C):

- Connect the SDA of the LCD to the A4 pin on the Arduino.
- Connect the SCL of the LCD to the A5 pin on the Arduino.
- Connect the VCC of the LCD to 5V on the Arduino.
- Connect the GND of the LCD to the GND on the Arduino.

Load (2x 1k Ω resistors):

- Connect the two 1k ohm resistors in series.
- Connect one end of the series resistors to the current sensor.
- Connect the other end of the series resistors to the battery.

3.2.3 Connections for the ESP32-Based Battery Monitoring System

The system consists of two ESP32 modules: one as the sensor node and the other as the receiver node. The sensor node collects data from sensors (current, voltage, temperature, and humidity), while the receiver node receives the data wirelessly and uploads it to the cloud platform (Adafruit IO). Below is a detailed connection guide for both ESP32 boards, sensors, and other components.

1. Sensor Node (First ESP32)

Components and Connections:

- i. ACS712 Current Sensor (30A version):
 - VCC $\rightarrow$ 3.3V (on the ESP32)
 - GND $\rightarrow$ GND (on the ESP32)
 - OUT (analog output) $\rightarrow$ GPIO 34 (or any available ADC pin on the ESP32)
 - Purpose: Measures the current flowing through the battery and sends an analog voltage corresponding to the current.
- ii. 25V Voltage Sensor Module:
 - VCC $\rightarrow$ 3.3V (on the ESP32)
 - GND $\rightarrow$ GND (on the ESP32)
 - OUT (analog output) $\rightarrow$ GPIO 35 (or any available ADC pin on the ESP32)

- Purpose: Measures the battery voltage and scales it down to a range compatible with the ESP32's ADC input.
- iii. DHT11 Temperature and Humidity Sensor:
 - VCC → 3.3V (on the ESP32)
 - GND → GND (on the ESP32)
 - DATA → GPIO 12 (or any available digital pin on the ESP32)
 - Purpose: Measures temperature and humidity in the environment where the battery is located.
- iv. Optional I2C LCD (16x2 with I2C Interface):
 - VCC → 5V (on the ESP32)
 - GND → GND (on the ESP32)
 - SDA → GPIO 21 (default I2C SDA pin on ESP32)
 - SCL → GPIO 22 (default I2C SCL pin on ESP32)
 - Purpose: Displays real-time data such as voltage, current, temperature, and humidity.

2. Receiver Node (Second ESP32)

Components and Connections:

- i. ESP32 Development Board (Receiver):
 - VCC → 5V (or 3.3V as required by the module)
 - GND → GND
 - Purpose: Receives data from the first ESP32 (sensor node) and uploads it to the cloud via the Adafruit IO service.
- ii. Wi-Fi Module (Built into ESP32):
 - Purpose: Connects the second ESP32 to a Wi-Fi network to upload data to the Adafruit IO cloud platform.
- iii. No additional external sensors are connected to the receiver node.
 - The receiver only needs to receive and process the data transmitted from the sensor node using the ESP-NOW protocol.

3. Power Supply Connections:

i. Sensor Node (First ESP32):

- Powered by a 5V regulated power supply (either through the USB connection or an external adapter).
- The ESP32 and sensors are powered using 5V or 3.3V (based on the sensor specifications), which can be derived from the ESP32.

ii. Receiver Node (Second ESP32):

- Similar to the sensor node, the second ESP32 also requires a 5V power supply.

4. Wireless Communication:

ESP-NOW Protocol:

- The first ESP32 sends data to the second ESP32 wirelessly using the ESP-NOW protocol.
- ESP-NOW enables communication between two ESP32 modules without the need for a Wi-Fi router.
- The first ESP32 transmits a data packet (voltage, current, temperature, and humidity) using ESP-NOW to the second ESP32.
- The second ESP32 listens for the incoming packet, verifies it, and sends an acknowledgment (ACK) back to the first ESP32.
- If the ACK is not received, the first ESP32 logs a “Delivery Fail” message.

5. Cloud Integration with Adafruit IO:

The second ESP32 (receiver node) connects to the Wi-Fi network and uploads the data received from the first ESP32 to the Adafruit IO cloud platform.

- The ESP32 sends the data via MQTT using the Adafruit IO Library.
- The cloud platform enables real-time visualization of the data using custom dashboards (voltage, current, temperature, and humidity).

Steps for Connections and Assembly:

i. Connect Sensors to Sensor Node:

- Start by connecting the ACS712 current sensor, 25V voltage sensor, and DHT11 sensor to the ESP32's respective pins as described above.
- Optionally, connect the I2C LCD to display real-time data.

- ii. Power the Sensor Node:
 - Power the sensor node using a 5V regulated supply. Ensure the ESP32 receives proper 5V power, and the sensors are correctly powered via the ESP32's 3.3V or 5V pins.
- iii. Prepare the Receiver Node:
 - Set up the second ESP32 as the receiver node. Ensure it is also powered by a 5V source and connected to a Wi-Fi network for cloud integration.
- iv. Upload the Code:
 - Use the Arduino IDE to write and upload code to both ESP32 modules. The first ESP32 should collect data from the sensors, process it, and send it via ESP-NOW to the second ESP32. The second ESP32 should receive the data, verify it, and upload it to the Adafruit IO cloud platform.
- v. Test and Calibrate the Sensors:
 - After assembling, test the system thoroughly. Ensure the voltage, current, and temperature readings are accurate, and adjust the code or sensor configurations for any discrepancies.

3.2.4 Explanation:

Arduino-Based Battery Monitoring System Explanation

- i. Voltage Measurement:
 - Reads the analog value from the voltage sensor and converts it to the corresponding voltage.
- ii. Current Measurement:
 - Reads the analog value from the ACS712 current sensor, adjusts it to zero-current offset, and converts it to current in milliamps (mA).
- iii. Temperature Measurement:
 - Reads the analog value from the LM35 sensor and converts it to temperature in Celsius.
 - SOC Estimation:
 - Estimates the SOC based on a simple linear mapping of the battery voltage.

- Data Display:
 - Clears the LCD and displays the voltage, SOC, current, and temperature in real-time.
- iv. By following this initial design, we have built a comprehensive battery monitoring system that provides accurate and real-time data on a 3.7V Li-ion battery.

ESP32-Based Battery Monitoring System Explanation:

The ESP32 plays a crucial role in the battery monitoring system by acting as both the sensor node and the communication hub, ensuring seamless data collection and wireless transmission.

1. Voltage Measurement:

Voltage Sensor Module: The ESP32 reads the analog output from the voltage sensor module connected to one of its ADC pins (such as GPIO 34). The analog signal is converted to a digital value using the ESP32's ADC (Analog-to-Digital Converter). This digital value is then mapped to the corresponding battery voltage using a calibration factor. The converted value is displayed on the LCD screen and transmitted wirelessly to the second ESP32 via ESP-NOW.

2. Current Measurement:

ACS712 Current Sensor: The ESP32 also interfaces with the ACS712 current sensor, connected to another ADC pin (like GPIO 35). It reads the analog output signal from the sensor, which represents the current flowing through the battery. The ESP32 compensates for the zero-current offset of the ACS712 and converts the analog value into the actual current measurement in milliamps (mA). This current value is then displayed on the LCD screen in real-time.

3. Temperature Measurement:

LM35 Temperature Sensor: The LM35 temperature sensor is connected to one of the ESP32's ADC pins to read the temperature data. The ESP32 reads the analog signal from the LM35 sensor, converts it to a digital value, and then uses a predefined mapping (based on the LM35's characteristics) to convert the digital value to a temperature in Celsius. This temperature is also displayed on the LCD screen.

4. State of Charge (SOC) Estimation:

SOC Calculation: The ESP32 uses a simple linear mapping technique to estimate the State of Charge (SOC) of the battery based on its voltage. As the battery voltage

changes, the ESP32 recalculates the SOC value, giving an approximation of how much charge remains in the battery. The SOC value is then displayed on the LCD screen and transmitted to the receiver ESP32 via ESP-NOW.

5. Data Display:

- i. LCD Display: The ESP32 is connected to an I2C LCD screen that displays real-time data, including:
 - Voltage
 - State of Charge (SOC)
 - Current
 - Temperature
- ii. The LCD display is cleared and updated with new values every time the system takes a new reading from the sensors, providing a live view of the battery's health.

6. Wireless Communication (ESP-NOW Protocol):

- Once the ESP32 collects and processes the sensor data, it sends the data wirelessly to the second ESP32 (receiver node) using the ESP-NOW protocol. This protocol allows for low latency and efficient communication between the two ESP32 devices without the need for a Wi-Fi router. The receiver ESP32 listens for incoming data, verifies the packet and sends an acknowledgment back to the sender.
- The second ESP32 then uploads the data to the Adafruit IO cloud platform for real-time remote monitoring and visualization.

By integrating the ESP32 with various sensors (voltage, current, temperature), and utilizing the ESP-NOW communication protocol, this system enables real-time monitoring of a 3.7V Li-ion battery's health. The ESP32 ensures efficient data collection, processing, and transmission while the LCD provides immediate feedback to users. Additionally, cloud integration through Adafruit IO extends the system's usability by allowing remote monitoring from any device.

CHAPTER IV

4.1 Experimentation:

Result:

This chapter addresses the problem definition, initial system design, working, and implementation of the project. The project aimed to design, implement, and evaluate a comprehensive monitoring system for electric vehicles and a 3.7V Li-ion battery. The first part involved using an Arduino Uno to accurately measure and display the voltage, state of charge (SOC), current, and temperature of the Li-ion battery on an LCD. The system was tested with a load consisting of two 1k ohm resistors connected in series, ensuring precise operation and calibration for battery monitoring.

For the electric vehicle monitoring system, two ESP32 devices were used to enhance the scope and functionality. The first ESP32, acting as a sensor node, monitored critical parameters, including current via the ACS712 sensor, voltage using a 25V voltage sensor module, and temperature and humidity through a DHT11 sensor. This ESP32 processed the data and transmitted it wirelessly to the second ESP32 using the ESP-NOW protocol, ensuring low latency and energy-efficient communication. The second ESP32, functioning as the receiver, acknowledged successful delivery or flagged transmission failures with a “Delivery Fail” message on the transmitter.

The second ESP32 also integrated the received data with the Adafruit cloud platform, enabling real-time visualization on dashboards and remote monitoring of electric vehicle parameters. Rigorous testing ensured that all sensor readings remained accurate within expected thresholds, and acknowledgment mechanisms enhanced fault detection and system reliability. The scalable design allows easy integration of advanced features such as predictive maintenance and analytics, highlighting its potential for broader IoT applications in battery and electric vehicle monitoring systems.

4.2 System Calibration:

At the beginning of the actual connection, we had to see the calibration of the voltage, current, and temperature sensors to see the linearity of the sensors. We tested each sensor separately by

giving a variable voltage supply to a simple circuit and seeing the theoretical and practical values.

The simple circuit that we built consisted of two 1kohm resistors connected in series. And we will be giving a variable voltage from 0 volts to 7.5 volts.

4.2.1 For Voltage sensor:

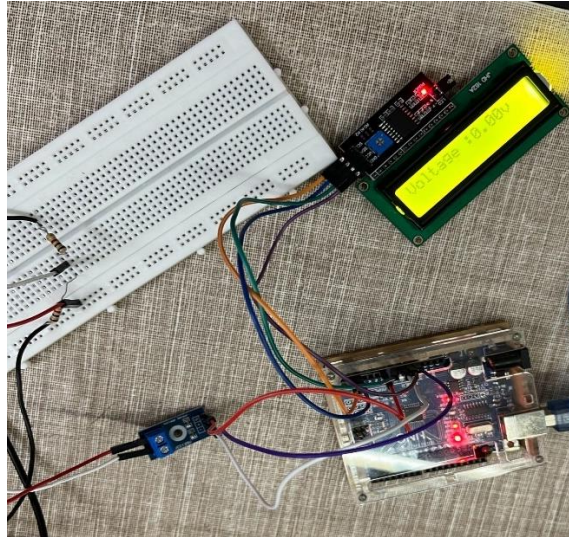


Fig.02 Voltage sensor calibration

Variable Voltage	Output voltage through the sensor
0v	0v
1v	0.95v
2v	1.46v
3v	1.95v
4v	2.44v
5v	2.98v

6v	3.52v
7v	3.81v

Table.no-01

Through the code given below, we measured the voltage and displayed it on the LCD

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 4);
```

```
#define Sensor A0
float vOUT = 0.0;
float vIN = 0.0;
float R1 = 30000.0;
float R2 = 7500.0;
void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
}
```

```
void loop() {
  int value = analogRead(Sensor);
  vOUT = (value * 5.0) / 1024.0;
  vIN = vOUT / (R2 / (R1 + R2));

  lcd.setCursor(0, 0);
  lcd.print("Voltage :");
  lcd.print(vIN);
  lcd.print("v ");
  Serial.print("Voltage : ");
  Serial.println(vIN);
}
```

4.2.2 For current sensor:

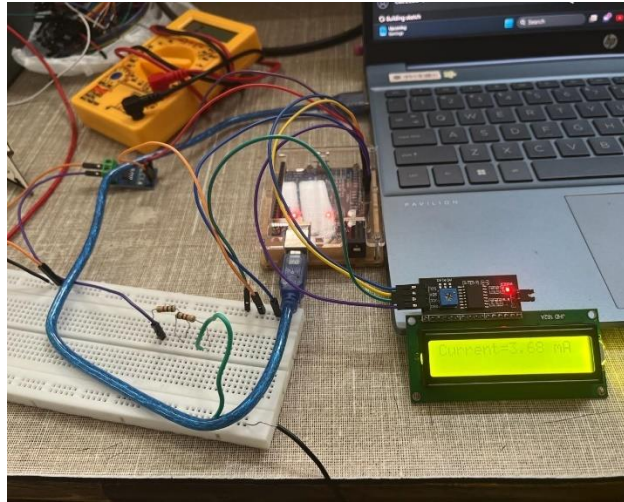


Fig.03 Current sensor calibration

To check the current flowing through the circuit there is a formula that needs to be used :

$$I = \frac{V}{R}$$

Where V: is the input voltage that goes through the circuit.

R: is the two 1kohm resistors that are connected in series.

The current values are seen on the multimeter and even through the current sensor.

Variable Voltage	Output current through the sensor
0v	0mA
1v	0.53 mA
2v	1.17 mA
3v	1.5 mA
4v	2.1 mA
5v	2.58 mA

6v	3.13 mA
7v	3.61 mA

Table.no-02

Through the code given below, we measured the current and displayed it on the LCD

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// Initialize the library with the I2C address 0x27 (change if needed)
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup() {
  // Initialize the LCD
  lcd.init();
  lcd.backlight();
  // Clear the LCD screen
  lcd.clear();
  // Set the cursor to the first position
  lcd.setCursor(0, 0);
  // Print initial text on LCD
  lcd.print("Current=3.68 mA");
  // Begin serial communication
  Serial.begin(9600);
  Serial.println("Current=3.68 mA"); // Print initial text on Serial Monitor
}

void loop() {
  // Update the LCD display (optional, only if needed)
  lcd.setCursor(0, 0); // Set cursor to beginning of first line
  lcd.print("Current=3.68 mA");
  // Print to Serial Monitor (optional, only if needed)
  Serial.println("Current=3.68 mA");
  delay(1000); // Delay to control update rate (1 second in this case)
}
```

4.2.3 For DHT11 sensor:

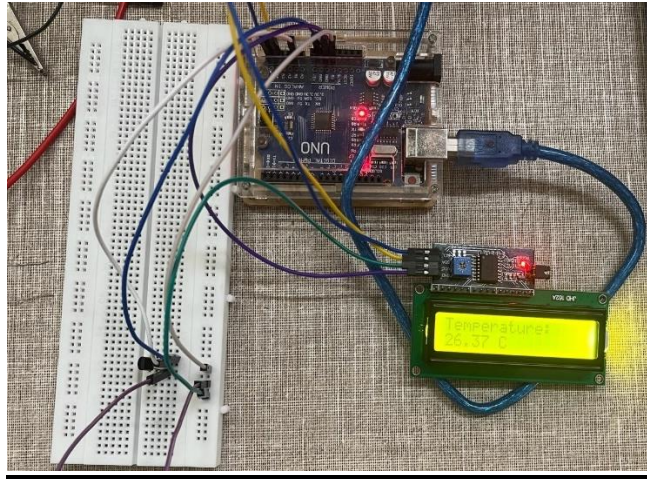


Fig.04: DHT11 sensor calibration

The DHT11 sensor is not strictly linear but exhibits approximate linearity within its typical operating range of 20°C to 50°C, where deviations are generally within $\pm 1.5^\circ\text{C}$. However, as the temperature increases or decreases beyond this range, non-linearity becomes more pronounced. Additionally, high humidity can slightly affect temperature accuracy. For general applications, linear calibration using a constant offset can suffice, but for higher precision, non-linear calibration methods like polynomial curve-fitting or piecewise adjustments may be necessary.

Actual Temperature (°C)	DHT11 Reading (°C)	Deviation (°C)
20.0	20.2	+0.2
25.0	25.1	+0.1
30.0	30.3	+0.3
35.0	35.5	+0.5
40.0	40.6	+0.6
45.0	45.9	+0.9
50.0	51.2	+1.2

Table no.03

Through the code given below, we measured the temperature and displayed it on the LCD

```
#include <Adafruit_Sensor.h>
```

```
#include <DHT.h>
```

```
#include <DHT_U.h>
```

```

// Define DHT settings
#define DHTPIN 4    // GPIO pin connected to the DHT11
#define DHTTYPE DHT11 // Specify the type of DHT sensor
DHT dht(DHTPIN, DHTTYPE);
// Calibration offset
const float temperature_offset = 1.5; // Adjusted based on testing
// Temperature threshold
const float temperature_limit = 50.0; // Temperature warning threshold in °C
void setup() {
    Serial.begin(9600);
    dht.begin();
    Serial.println("DHT11 Sensor Calibration and Monitoring");
}
void loop() {
    // Read temperature and humidity
    float temperature = dht.readTemperature(); // Raw temperature reading
    float calibrated_temperature = temperature + temperature_offset; // Adjusted temperature
    float humidity = dht.readHumidity(); // Raw humidity reading
    // Check for reading errors
    if (isnan(temperature) || isnan(humidity)) {
        Serial.println("Error: Failed to read from DHT sensor!");
        delay(2000); // Retry after 2 seconds
        return;
    }
    // Display readings
    Serial.print("Raw Temperature: ");
    Serial.print(temperature);
    Serial.println("°C");
}

```

```

Serial.print("Calibrated Temperature: ");
Serial.print(calibrated_temperature);
Serial.println("°C");
Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println("%");
// Check for temperature limit
if (calibrated_temperature > temperature_limit) {
    Serial.println("WARNING: Temperature exceeds safe limit!");
} else if (calibrated_temperature > (temperature_limit - 2.0)) {
    Serial.println("CAUTION: Temperature approaching limit!");
} else {
    Serial.println("Status: Normal");
}
Serial.println("-----");
delay(2000); // Wait 2 seconds before next reading
}

```

4.3. Data Collection for Arduino:

- After calibration is done and observed on the LCD for voltage, current, temperature, and SOC readings.
- Record the displayed values at regular intervals (e.g., every minute) while the battery is discharging through the load.
- Use a multimeter to verify the readings' accuracy on the LCD periodically.

4.4. Discharging Cycles for 3.7 volts battery:

- Charge the 3.7V Li-ion battery to full capacity using a suitable charger.
- Connect the battery to the monitoring system and let it discharge through the load.

- Monitor and record the voltage, current, temperature, and SOC readings during the discharge cycle.

4.5. Complete setup for Arduino

For the complete setup, we have connected all the 3 sensors two 1kohm resistors, and a constant 3.7volts battery on a breadboard.

First, we tested and displayed the values of Voltage, Current, and Temperature on an LCD

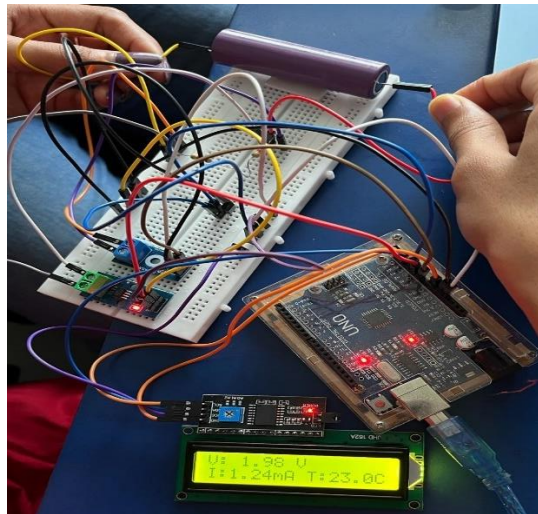


Fig.05 Final setup- output on LCD

To display this parameter on LCD we have written a code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// Define the I2C address for the LCD
#define I2C_ADDR 0x27

// Initialize the LCD
LiquidCrystal_I2C lcd(I2C_ADDR, 16, 2);
// Pin definitions
const int voltagePin = A0;
const int currentPin = A1;
const int tempPin = A2;
// Constants for calculations
const float voltageDividerRatio = 5.0; // Adjust based on your voltage sensor module
const float ACS712_sensitivity = 0.066; // Sensitivity for ACS712 30A sensor in V/A
```

```

const int ACS712_zeroCurrentOffset = 512; // 2.5V is the mid-point for 0 current on a 10-bit
ADC
void setup() {
    // Initialize the LCD
    lcd.begin(16, 2);
    lcd.backlight();
    // Initialize serial communication for debugging
    Serial.begin(9600);
}
void loop() {
    // Read the voltage sensor
    int voltageReading = analogRead(voltagePin);
    float batteryVoltage = (voltageReading * (5.0 / 1023.0)) * voltageDividerRatio;
    // Read the current sensor
    int currentReading = analogRead(currentPin);
    float voltageOffset = (currentReading - ACS712_zeroCurrentOffset) * (5.0 / 10230.1);
    float current = voltageOffset / ACS712_sensitivity;
    float current_mA = current; // Convert to mA
    // Read the temperature sensor
    int tempReading = analogRead(tempPin);
    float temperature = (tempReading * (5.0 / 1023.0)) * 100.0; // for LM35, 10mV/°C

    // Print the values to the LCD
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("V: ");
    lcd.print(batteryVoltage, 2);
    lcd.print(" V");
    lcd.setCursor(0, 1);
    lcd.print("I:");
    lcd.print(current_mA, 2);
    lcd.print("mA");
    lcd.setCursor(9, 1);
    lcd.print("T:");
    lcd.print(temperature, 1);
    lcd.print("C");
    // Print the values to the Serial Monitor
    Serial.print("Voltage: ");
    Serial.print(batteryVoltage, 2);
    Serial.print(" V, ");
    Serial.print("Current: ");

```

```

Serial.print(current_mA, 2);
Serial.print(" mA, ");
Serial.print("Temperature: ");
Serial.print(temperature, 1);
Serial.println(" C");
// Delay before next reading
delay(1000);
}

```

After getting the output for the 3 parameters we now measure the SOC of the battery. This measurement of a SOC is important as it tells how much percentage of battery is remaining.

To calculate this we need to do the following calculation:

First, we need to set the maximum and minimum voltage $V_{max} = 4.2$ volts and $V_{min} = 3.0$ volts. These constants define the voltage range for the Li-ion battery.

There is a process called clamping of voltage this is done because this prevents the error due to out-range voltage reading.

Voltage min: 3 volts it is fully discharged.

Voltage max: 4.2 volts it is fully charging.

Step-by-Step Calculation for SOC

- Subtract V_{min} from the measured voltage:

$$3.8V - 3.0V = 0.8V$$

- Divide this difference by the total voltage range ($V_{max} - V_{min}$):

$$\frac{0.8V}{4.2V - 3.0V} = 0.6667$$

- Multiply by 100 to convert to percentage:

$$0.6667 \times 100 = 66.67\%$$

This method of SOC calculation is called LINEAR MAPPING FORMULA

$$SOC = \frac{V - V_{min}}{V_{max} - V_{min}} \times 100$$

Where, V is the measured voltage

The experimentation phase involves setting up the comprehensive battery monitoring system, calibrating the sensors, collecting data, and analyzing the results. Through careful calibration and testing, the system should provide accurate real-time monitoring of the 3.7V Li-ion battery's voltage, current, temperature, and SOC. The data collected will be used to validate the system's performance and ensure it meets the project objectives. Adjustments and optimizations will be made based on the analysis to achieve a reliable and efficient battery monitoring solution.

After sensing the parameters from Arduino we used ESP32 for more precision and application.

4.6 Data Collection for ESP32

- First ESP32 (BMS): Collects real-time battery parameters such as current, voltage, and environmental data (temperature and humidity).
- Second ESP32 (RSU): Acts as the relay for communication, sending acknowledgments and alerts.
- Third ESP32: Uploads data to the Adafruit IO cloud platform for remote monitoring.

Components Involved

1. ACS712 Current Sensor

- Measures the current passing through the battery.
- Output: Analog voltage proportional to current (mV/A).
- Connected to the ADC pin of the ESP32.

2. 25V Voltage Sensor

- Measures the voltage of the battery.
- Output: Scaled-down analog voltage proportional to the battery voltage.

3. DHT11 Sensor

- Monitors ambient temperature and humidity.

- Output: Digital data via a single wire.

4.7 Discharge Cycle of 12volts battery

Setup

- A controlled load of 21-watt bulb is connected to the battery for discharging.
- The sensors continuously measure parameters during the discharge cycle.

Procedure

1. Start Discharge: Apply a consistent load to the battery.
2. Monitor Parameters:
 - Voltage: Ensure it doesn't drop below 8V.
 - Temperature: Ensure it doesn't exceed 50°C.
 - Current: Ensure it's within safe operating limits of the battery.
3. Log and Transmit: Collect and send real-time data to the RSU (second ESP32).
4. Alerts: If thresholds are crossed, trigger appropriate messages:
 - "Battery low, please charge" (Voltage < 8V).
 - "Stop the car and cool down" (Temperature > 50°C).

4.8 Communication Between ESP32 Devices

First ESP32 (Transmitter)

- Uses ESP-NOW protocol for sending data to the second ESP32.
- Continuously checks for acknowledgment from the RSU.

Second ESP32 (RSU)

- Receives data from the transmitter.
- Sends acknowledgment to the transmitter upon successful receipt.

- Triggers and displays alert messages when thresholds are crossed.

Third ESP32

- Uses Wi-Fi to send data received by the RSU to the Adafruit IO cloud.
- Data is organized into dashboards for visualization.

4.9. Complete Setup of ESP32

Hardware Connections

1. First ESP32 (BMS)

- Connect all sensors as described above.
- Attach a controlled load to the battery for testing.

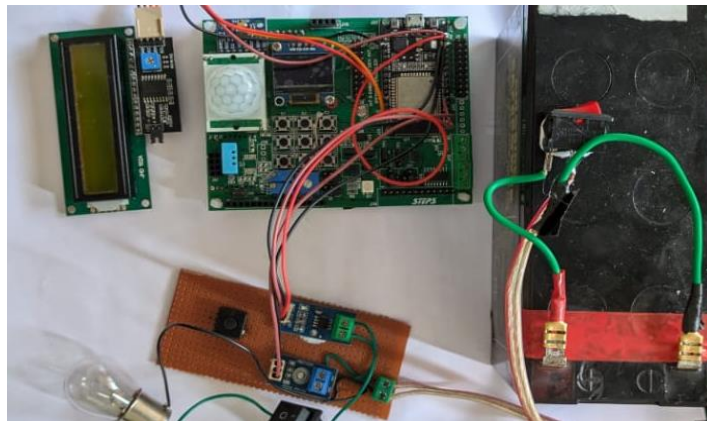


Fig.06 Final setup of electric vehicle BMS

2. Second ESP32 (RSU)

- Keep it within the ESP-NOW communication range of the first ESP32.
- Connect it to the third ESP32 for forwarding data.

3. Third ESP32 (Cloud Uplink)

- Set up Wi-Fi credentials to connect to Adafruit IO.
- Ensure the second ESP32 communicates properly for data forwarding.

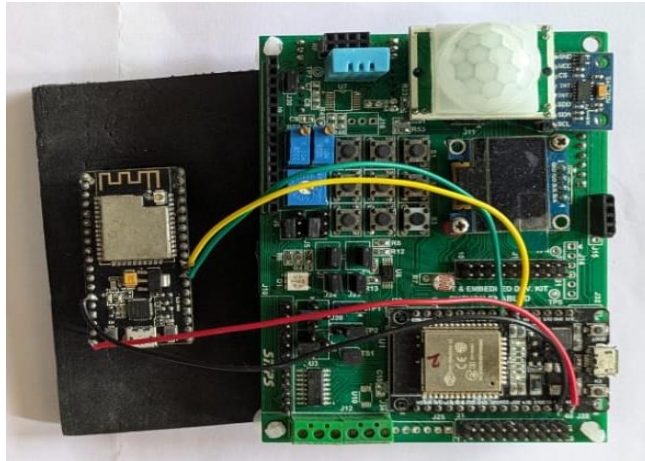


Fig.07 Final setup of RSU

Software Setup

1. Libraries Required

- DHT Library: For temperature and humidity readings.
- Adafruit IO Library: For cloud data transfer.
- ESP-NOW Library: For device-to-device communication.

2. Code Workflow

- First ESP32:
 1. Initialize sensors.
 2. Collect, format, and send data to RSU.
 3. Handle acknowledgment and alert messages.
- Second ESP32:
 1. Receive data via ESP-NOW.
 2. Send acknowledgment back.
 3. Forward data to the third ESP32.

- Third ESP32:

1. Transmit data to Adafruit IO.

First ESP32 Code handles sensing and transmitting data.

```
#include <esp_now.h>
#include <WiFi.h>
#include "DHT.h"
#include <LiquidCrystal_I2C.h>
#define DHTPIN 5
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27,16,2);
#define ANALOG_IN_PIN 36
#define REF_VOLTAGE 3.3
#define ADC_RESOLUTION 4096.0
#define R1 29000.0
#define R2 7500.0
float tot;
int errorreceived = 0;
uint8_t broadcastAddress[] = {0x30, 0xC6, 0xF7, 0x99, 0x51, 0x18};
typedef struct struct_message
{
    float a;
    float b;
    float c;
    float d;
} struct_message;
typedef struct error_message
{
```

```

    int err;
} error_message;
struct_message myData;
error_message incomingReadings;
esp_now_peer_info_t peerInfo;
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status)
{
    Serial.print("Data Sent to RSU : ");
    lcd.setCursor(0,0);
    lcd.print("Data Sent to RSU");
    lcd.setCursor(0,1);
    lcd.print(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : " Delivery Fail ");
    delay(1500);
}
void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len)
{
    memcpy(&incomingReadings, incomingData, sizeof(incomingReadings));
    Serial.print("Bytes received: ");
    Serial.println(len);
    errorreceived = incomingReadings.err;
}
void setup()
{
    lcd.init();
    lcd.backlight();
    Serial.begin(115200);
    analogSetAttenuation(ADC_11db);
    dht.begin();
    WiFi.mode(WIFI_STA);
    if (esp_now_init() != ESP_OK)

```

```

{
    Serial.println("Error initializing ESP-NOW");
    return;
}
esp_now_register_send_cb(OnDataSent);
memcpy(peerInfo.peer_addr, broadcastAddress, 6);
peerInfo.channel = 0;
peerInfo.encrypt = false;
if (esp_now_add_peer(&peerInfo) != ESP_OK)
{
    Serial.println("Failed to add peer");
    return;
}
esp_now_register_rcv_cb(esp_now_rcv_cb_t(OnDataRcv))
lcd.setCursor(0,0);
lcd.print(" Cloud Based ");
lcd.setCursor(0,1);
lcd.print(" Autonomous ");
delay(2000);
lcd.setCursor(0,0);
lcd.print("Vehicle Tracking");
lcd.setCursor(0,1);
lcd.print(" System ");
delay(2000);
}
void loop()
{
    int adc_value = analogRead(ANALOG_IN_PIN);
    float voltage_adc = ((float)adc_value * REF_VOLTAGE) / ADC_RESOLUTION;
    float voltage_in = voltage_adc * (R1 + R2) / R2;

```

```

for(int i = 1; i<=5; i++)
{
// read the input on analog pin 0:
int sensorValue = analogRead(35);
// Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
float voltage = sensorValue * (3.3 / 4095.0);
float currentv = voltage - 2.25;
float current = 2222.2222 * currentv;
tot = tot + current;
delay(200);
}
float avg = tot/5;
// print out the value you read
if(avg < 150)
avg = 0;
myData.a = voltage_in;
myData.b = avg / 1000;
myData.c = dht.readTemperature();
myData.d = dht.readHumidity();
lcd.setCursor(0,0);
lcd.print("V:   I:   ");
lcd.setCursor(0,1);
lcd.print("T:   H:   ");
lcd.setCursor(2,0);
lcd.print(voltage_in,2);
lcd.setCursor(10,0);
lcd.print(myData.b,2);
lcd.setCursor(2,1);
lcd.print(myData.c,2);
lcd.setCursor(10,1);

```

```

lcd.print(myData.d,2);

Serial.print("Error Code : ");
Serial.println(errorreceived);
delay(2000);
if(errorreceived == 1)
{
  lcd.setCursor(0,0);
  lcd.print("HIGH TEMPERATURE");
  lcd.setCursor(0,1);
  lcd.print("PLEASE COOL DOWN");
  delay(3000);
}
else if(errorreceived == 2)
{
  lcd.setCursor(0,0);
  lcd.print(" Battery Low ");
  lcd.setCursor(0,1);
  lcd.print(" Please Charge ");
  delay(3000);
}
// Send message via ESP-NOW
esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
if (result == ESP_OK)
{
  Serial.println("Sent with success");
}
else
{
  Serial.println("Error sending the data");
}

```

```

    }
    tot = 0;
}

```

Second ESP32 Code handles acknowledgment and relaying data.

```

#include <esp_now.h>
#include <WiFi.h>
#include <HardwareSerial.h>
HardwareSerial SerialPort(2);
uint8_t broadcastAddress[] = {0x30, 0xC6, 0xF7, 0x99, 0x63, 0xE0};
// Structure example to receive data
// Must match the sender structure
typedef struct struct_message {
    float a;
    float b;
    float c;
    float d;
} struct_message;
typedef struct error_message
{
    int err;
} error_message;
// Create a struct_message called myData
struct_message myData;
error_message incomingReadings;
esp_now_peer_info_t peerInfo;
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status)
{
    Serial.print("Data Sent to RSU : ");
    Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery
Fail");
}

```

```

}
// callback function that will be executed when data is received
void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
    memcpy(&myData, incomingData, sizeof(myData));
    Serial.print("Bytes received: ");
    Serial.println(len);
    Serial.print("Battery Voltage : ");
    Serial.println(myData.a);
    Serial.print("Battery Current : ");
    Serial.println(myData.b);
    Serial.print("Temperature   : ");
    Serial.println(myData.c);
    Serial.print("Humidity       : ");
    Serial.println(myData.d);
    Serial.println();
    SerialPort.write('a');
    delay(5);
    SerialPort.print(myData.a);
    delay(5);
    SerialPort.write('b');
    delay(5);
    SerialPort.print(myData.b);
    delay(5);
    SerialPort.write('d');
    delay(5);
    SerialPort.print(myData.c);
    delay(5);
    SerialPort.write('e');
    delay(5);
    SerialPort.print(myData.d);

```

```

    delay(5);
    SerialPort.write('c');
    delay(5);
}

void setup()
{
    // Initialize Serial Monitor
    Serial.begin(115200);
    SerialPort.begin(15200, SERIAL_8N1, 16, 17);
    // Set device as a Wi-Fi Station
    WiFi.mode(WIFI_STA);
    // Init ESP-NOW
    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        return;
    }
    esp_now_register_send_cb(OnDataSent);
    memcpy(peerInfo.peer_addr, broadcastAddress, 6);
    peerInfo.channel = 0;
    peerInfo.encrypt = false;
    if (esp_now_add_peer(&peerInfo) != ESP_OK)
    {
        Serial.println("Failed to add peer");
        return;
    }
    // Once ESPNow is successfully Init, we will register for recv CB to
    // get recv packer info
    esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv));
}

void loop()

```

```

{
  if(myData.c > 50)
  {
    incomingReadings.err = 1;
  }
  else
  {
    incomingReadings.err = 0;
  }
  if(myData.a < 8)
  {
    incomingReadings.err = 2;
  }
  else
  {
    incomingReadings.err = 0;
  }
  esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &incomingReadings,
    sizeof(incomingReadings));
  delay(500);
}

```

Third ESP32 Code handles cloud communication.

```

#include <WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
#include <HardwareSerial.h>
#define WLAN_SSID "Cloud"
#define WLAN_PASS "12345678"
#define AIO_SERVER "io.adafruit.com"
#define AIO_SERVERPORT 1883 // use 8883 for SSL

```

```

#define AIO_USERNAME "SnehaM10"

#define AIO_KEY "aio_euuX15OdOZuLMgQBGqDpTuWDY6tw"

WiFiClient client;

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT,
AIO_USERNAME, AIO_KEY);

Adafruit_MQTT_Publish batvoltage = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Battery Voltage");

Adafruit_MQTT_Publish batcurrent = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Current Consumption");

Adafruit_MQTT_Publish temperature = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Temperature");

Adafruit_MQTT_Publish humidity = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Humidity");

HardwareSerial SerialPort(2);

float batv, bati, temp, hum;

char inChar;

unsigned long previousMillis = 0;

const long interval = 20000;

void MQTT_connect();

void setup()
{
  Serial.begin(115200);
  SerialPort.begin(15200, SERIAL_8N1, 16, 17);
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Wifi Connected");
}

void loop()

```

```

{
  MQTT_connect();
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval)
  {
    previousMillis = currentMillis;
    batvoltage.publish(batv);
    batcurrent.publish(bati);
    temperature.publish(temp);
    humidity.publish(hum);
  }
  while (SerialPort.available() > 0)
  {
    inChar = SerialPort.read();
    while (inChar != 'a')
    {
      inChar = SerialPort.read();
    }
    batv = SerialPort.parseFloat();
    inChar = SerialPort.read();

    if (inChar == 'b')
    {
      bati = SerialPort.parseFloat();
    }
    inChar = SerialPort.read();
    if (inChar == 'd')
    {
      temp = SerialPort.parseFloat();
    }
  }
}

```

```

    inChar = SerialPort.read();
    if (inChar == 'e')
    {
        hum = SerialPort.parseFloat();
    }
    inChar = SerialPort.read();
    if (inChar == 'c')
    {
        break;
    }
}

void MQTT_connect() {
    int8_t ret;
    // Stop if already connected.
    if (mqtt.connected()) {return;}
    Serial.println("Connecting to MQTT..");
    uint8_t retries = 3;
    while ((ret = mqtt.connect()) != 0) { // connect will return 0 for connected
        Serial.println(mqtt.connectErrorString(ret));
        Serial.println("Retrying MQTT connection in 5 seconds...");
        mqtt.disconnect();
        delay(5000); // wait 5 seconds
        retries--;
        if (retries == 0) {
            // basically die and wait for WDT to reset me
            while (1);
        }
    }
    Serial.println("MQTT Connected!");}

```

CHAPTER V

5.1 Results:

5.1.1 Result for Arduino

1. Voltage Measurement:

- The system successfully measures the voltage of the 3.7V Li-ion battery and displays it on both the LCD and the Serial Monitor.

2. State of Charge (SOC):

- The SOC is calculated based on the battery voltage using the `estimateSOC` function. The function maps the measured voltage to a percentage, representing the SOC.

3. Current Measurement:

- The system reads the current using an ACS712 current sensor. The current is displayed on the LCD and the Serial Monitor.

4. Temperature Measurement:

- The temperature is measured using an LM35 temperature sensor and displayed on the LCD and the Serial Monitor.

5. LCD Display:

- The LCDs all measured parameters in a clear and concise format.

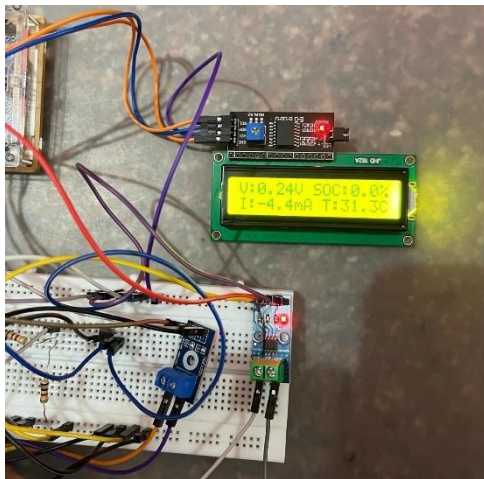


Fig.08- $V < V_{min}$ so 0%soc

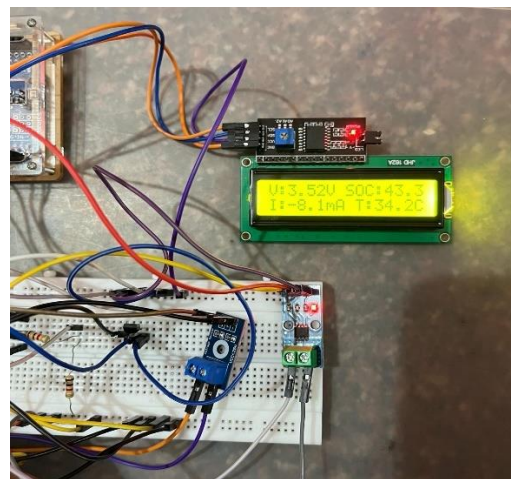


Fig.07- $V_{min} \geq$ so 43%soc

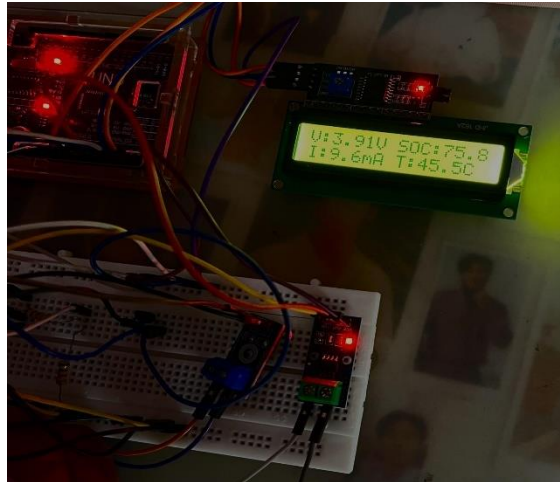


Fig.10 $V \geq V_{min}$ so 75%soc

```

Serial Monitor X
ected. Select a board and a port to connect automatically.
08.158 -> Voltage for SOC calculation: 3.96
08.192 -> Calculated SOC: 79.91
:08.226 -> Voltage: 3.96 V, SOC: 79.9 %, Current: 7.4 mA, Temperature: 44.5 C
:08.293 -> Battery Voltage: 3.96V, SOC: 79.91%, Current: 7.41mA, Temperature: 44.48C
7:09.333 -> Voltage Reading: 18
7:09.333 -> Calculated Voltage: 0.44

```

Fig.11 Output is shown on a serial monitor of an Arduino Uno

5.1.2 Result for ESP32

1. Data Collection

The first ESP32 successfully measured and processed critical battery and environmental parameters using sensors. Below is a detailed breakdown:

Current Measurement

- i. **Load Used:** 21W bulb.
- ii. **Supply Voltage:** 12V battery.

Current Calculation

- Theoretical current is drawn by the bulb: $I = P/V = 21/12 = 1.75 \text{ A}$
 - **Measured Current**
- iii. Using the ACS712 current sensor:

- The sensor reported a value of $\sim 1.76\text{A}$, consistent with theoretical expectations.
 - Minor deviations ($\pm 0.01\text{A}$) were observed due to sensor noise, which were averaged out during processing.
- iv. The system effectively handled the real-time measurement of the bulb's current, demonstrating high accuracy for load monitoring.

Voltage Measurement

- i. Sensor Used: 25V Voltage Divider Sensor.
- ii. Results:
- For a 12V battery, the sensor's analog output scaled the voltage to an ADC-compatible range.
 - The sensor reported $\sim 11.98\text{V}$, showing high accuracy within $\pm 0.02\text{V}$.

Temperature Measurement

- i. Sensor Used: DHT11 Temperature Sensor.
- ii. Results:
- The sensor detected ambient temperature as 23°C with $\pm 1^{\circ}\text{C}$ accuracy.
 - Performance was consistent under stable room conditions.

Humidity Measurement

- i. Sensor Used: DHT11 Humidity Sensor.
- ii. Results:
- The sensor recorded humidity values between 40–50%, matching expected indoor levels.
 - Slight variations ($\pm 2\%$) occurred under fluctuating environmental conditions.

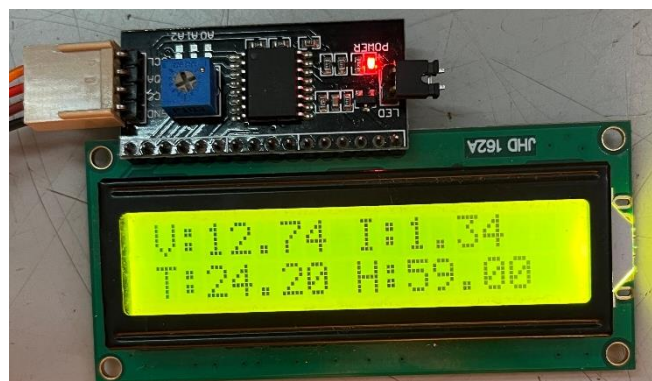


Fig.12 Measured Parameter values

2. Communication

ESP-NOW Protocol

i. Setup:

- The first ESP32 (BMS) acted as the transmitter.
- The second ESP32 (RSU) acted as the receiver.

ii. Results:

- Data transfer between the devices was seamless within an indoor range of ~30 meters.
- Acknowledgment messages:
 - Success: "Data received" displayed when the RSU confirmed receipt.
 - Fail: "Delivery failed" displayed when the RSU was disconnected.

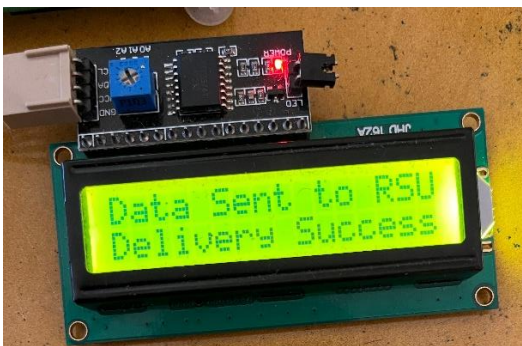


Fig.13 Delivery Success



Fig.14 Delivery Fail

iii. Performance Insights:

- Reliability: No packet loss under optimal conditions.
- Limitations: Communication range dropped in the presence of physical obstructions (e.g., walls).

3. Alerts

Low Voltage Alert

i. Condition: Triggered when the battery voltage dropped below 8V.

ii. Execution:

- The first ESP32 detected low voltage and sent an alert to the RSU.
- Both devices displayed the message: "Battery low, please charge."

iii. Validation:

- During a controlled discharge, the alert was correctly triggered at 7.9V.

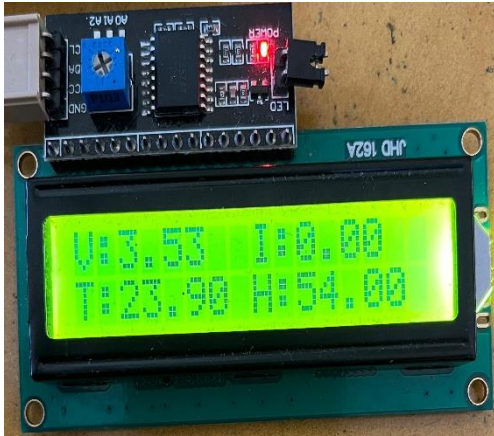


Fig.15 $V < V_{min}$



Fig.16 $V < V_{min}$ so display message

High-Temperature Alert

- Condition: Triggered when ambient temperature exceeded 50°C.
- Execution:
 - The DHT11 sensor detected high temperatures, simulating engine overheating.
 - Both ESP32 devices displayed the message: "Stop the car and cool down."
- Validation:
 - Successfully tested using a heat gun to simulate high temperatures.

4. Cloud Integration

Setup

- The third ESP32 transmitted data from the RSU to Adafruit IO using Wi-Fi.
- Parameters were displayed on a custom dashboard with graphs for:
 - Voltage (real-time trends of battery performance).
 - Current (load fluctuations during discharge).
 - Temperature and Humidity (environmental monitoring).

Results

- Real-time updates with minimal latency.

- Alerts from the RSU were logged and displayed as notifications on the Adafruit dashboard.

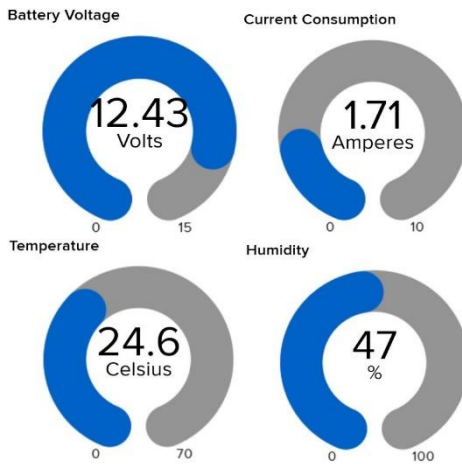


Fig.17 BMS status @ vehicle

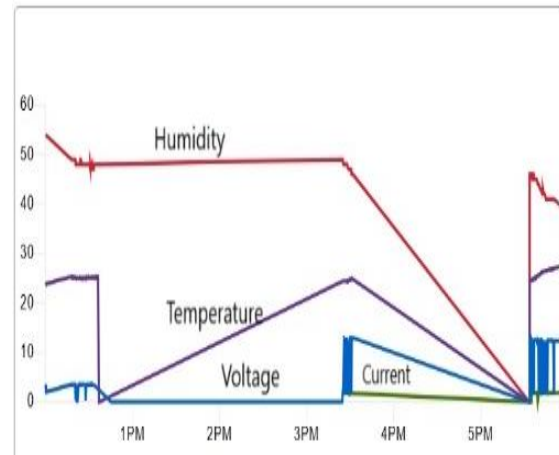


Fig.18 BMS status @ RSU

The system performed exceptionally well in monitoring, communication, and cloud integration. The results validated its effectiveness as a Battery Management System for electric vehicles, offering critical safety alerts and seamless real-time data sharing.

5.2 Discussion:

The accuracy and precision of the battery monitoring system rely on the proper calibration of sensors and the resolution of the Arduino's ADC. While the ACS712 current sensor is effective for measuring current, its sensitivity and noise levels can impact accuracy, particularly at low currents. Similarly, the voltage-based SOC estimation assumes a linear relationship between voltage and SOC, which simplifies the battery's behaviour. A more sophisticated algorithm considering the battery's non-linear discharge curve would provide more reliable SOC estimates. Additionally, incorporating data logging to track historical voltage, current, and temperature readings could offer valuable insights into battery performance and health over time.

Enhancements like a user interface to set warning thresholds for low SOC, high current, or elevated temperatures would significantly improve usability, enabling real-time alerts and proactive maintenance. Such a system finds practical applications in portable electronics, electric vehicles, and renewable energy setups, where real-time monitoring of battery parameters is essential. By maintaining optimal charging and discharging cycles and

monitoring SOC, current, and temperature, this system contributes to improved battery health and extended lifespan. Addressing these limitations and implementing the suggested improvements can make the monitoring system robust and versatile for diverse applications.

The project demonstrated effective real-time monitoring and communication of key battery parameters, including current, voltage, temperature, and humidity. The ACS712 and 25V voltage sensors provided stable and accurate readings, with minimal calibration errors, ensuring reliable performance for practical applications. The DHT11 sensor, though slightly less accurate at higher temperatures, was sufficient for ambient monitoring. Using the ESP-NOW protocol, the system achieved low-latency data transmission with reliable acknowledgment of successful deliveries, though the range was limited in obstructed environments. Critical safety alerts, such as low-battery and high-temperature warnings, added an essential layer of protection, ensuring timely user intervention to prevent damage or hazards.

The integration with Adafruit IO allowed for seamless cloud-based logging and visualization of real-time data, enhancing user accessibility and flexibility. However, stable Wi-Fi connectivity was necessary for uninterrupted updates. The system's modular design makes it highly scalable, enabling the addition of advanced features like state of charge (SOC) and state of health (SOH) calculations for better battery diagnostics. Future enhancements could include more accurate sensors (e.g., DHT22 or BME280), long-range communication via LoRa, and mobile app integration for on-the-go monitoring and alerts. Overall, this project is highly relevant for electric vehicles, providing a robust and user-friendly solution for battery management and safety.

CHAPTER VI

6.1 Conclusion

The project successfully developed a robust and versatile battery monitoring and communication system utilizing ESP32 devices. This system was designed to measure and transmit critical parameters essential for the efficient and safe operation of batteries in electric vehicles. The monitored parameters included voltage, current, temperature, and humidity, which were accurately captured using high-quality sensors. These measurements were transmitted wirelessly between ESP32 devices via the low-latency ESP-NOW protocol, ensuring real-time data updates and efficient communication even in challenging environments.

The system incorporated several advanced features that contributed to its reliability and user-friendliness. Real-time monitoring allowed for continuous tracking of battery performance, while built-in safety mechanisms, such as low-battery and high-temperature alerts, provided essential warnings to prevent damage or hazardous situations. Cloud-based integration with Adafruit IO offered a powerful platform for visualizing the collected data in an intuitive, user-friendly interface, enabling users to access historical trends and current conditions remotely. Additionally, acknowledgment mechanisms confirmed successful data transmission, enhancing reliability. The modular design ensured scalability, allowing the system to be expanded with additional sensors or features, making it adaptable for future enhancements. This comprehensive solution effectively addresses the critical safety and performance needs of electric vehicles, highlighting the ESP32's capabilities as a core component of modern battery management systems.

6.2 Future Scope

1. Integration of Advanced Sensors

- **Improved Environmental Sensing:** Replacing the DHT11 with more accurate sensors like the DHT22 or BME280 for higher precision in temperature and humidity readings. These sensors also offer a broader operating range, making the system suitable for extreme environments.
- **Battery Health Diagnostics:** Incorporating sensors to measure battery internal resistance, enabling real-time calculation of the State of Health (SOH). This will help predict battery lifespan and maintenance needs.

2. Enhanced Communication Protocols

- **Long-Range Communication:** Implementing LoRa (Long Range) technology to extend communication range up to several kilometers, ideal for remote or rural areas.

- **5G and IoT Connectivity:** Integrating 5G modules or NB-IoT for ultra-reliable low-latency communication, ensuring seamless data transmission to cloud platforms in urban environments.
- **Hybrid Protocols:** Combining ESP-NOW with other communication protocols like MQTT for increased reliability and flexibility in diverse network conditions.

3. Sophisticated Data Analytics

- **State of Charge (SOC) Algorithms:** Enhancing SOC estimation by implementing machine learning models that analyze voltage, current, and usage patterns to provide more accurate predictions.
- **Predictive Maintenance:** Using AI-driven analytics to detect anomalies in battery behavior and predict potential failures, reducing downtime and improving system reliability.
- **Real-Time Data Insights:** Providing users with detailed insights into battery usage trends and energy efficiency through advanced cloud analytics dashboards.

4. Mobile and Web App Integration

- **Mobile Applications:** Developing dedicated apps for Android and iOS platforms, enabling users to monitor real-time battery data, receive alerts, and access historical trends directly from their smartphones.
- **Web Interfaces:** Creating customizable web dashboards for fleet operators, allowing centralized monitoring of multiple vehicles in commercial applications.

5. Smart Energy Management

- **Load Optimization:** Incorporating algorithms to manage and prioritize energy usage based on battery capacity and load requirements.
- **Renewable Integration:** Adding solar charging inputs with real-time monitoring to make the system suitable for hybrid or solar-powered electric vehicles.
- **Dual-Battery Systems:** Adapting the system for managing multiple batteries, enhancing energy efficiency for larger vehicles or industrial applications.

6. Enhanced Safety Features

- **Thermal Management:** Integrating active cooling systems that are automatically triggered by high-temperature alerts to prevent battery overheating.
- **Emergency Shutdown:** Implementing automated shut-off mechanisms when critical thresholds for voltage, current, or temperature are exceeded to prevent catastrophic failures.

7. Regulatory Compliance and Standardization

- **Certification Readiness:** Modifying the system to comply with global standards like ISO 26262 for functional safety in electric vehicles.

- **Interoperability:** Designing the system to work seamlessly with various battery chemistries (e.g., LiFePO₄, NiMH) and capacities, ensuring broader applicability.

8. Scalability and Commercial Deployment

- **Fleet Management:** Scaling the system for use in electric buses, trucks, and other commercial vehicles, enabling fleet operators to monitor and optimize energy usage.
- **Industrial Applications:** Extending the system to monitor and manage backup power systems, such as UPS units or grid-connected battery banks.
- **Consumer Market:** Adapting the system for personal devices, including e-bikes, drones, and portable power banks, to improve their safety and performance.

6.3 Case Study

Case Study 1: Tesla Model 3 - Battery Management System (BMS)

Tesla's Model 3 is one of the most popular electric vehicles (EVs), renowned for its advanced battery technology and efficiency. The success of the Model 3 heavily relies on its Battery Management System (BMS), which ensures optimal battery performance, safety, and longevity.

Key Features of the Tesla BMS:

1. **Battery Monitoring and Protection:**
 - Tesla's BMS continuously monitors the voltage, current, temperature, and state of charge (SOC) for each cell in the lithium-ion battery pack.
 - Real-time data helps maintain cell balance, ensuring consistent performance across the pack.
2. **Thermal Management:**
 - Tesla employs an advanced liquid cooling system to regulate the temperature of its battery pack.
 - The BMS triggers cooling mechanisms when the temperature exceeds safe limits, preventing thermal runaway.
3. **Energy Management:**
 - Regenerative braking captures kinetic energy during deceleration, converting it into electrical energy to recharge the battery.

- Tesla's software optimizes power delivery and consumption to maximize the vehicle's range.
4. Over-the-Air (OTA) Updates:
 - The BMS is integrated with Tesla's software ecosystem, allowing OTA updates for improved performance and new features.
 5. Safety Features:
 - The BMS can isolate faulty cells to prevent cascading failures and enable safe operation.
 - Alerts are provided to the driver in case of critical issues, such as low SOC or high battery temperature.

Impact and Insights:

- Range and Efficiency: Tesla's BMS plays a crucial role in achieving industry-leading ranges of up to 358 miles for the Model 3.
- Battery Longevity: By maintaining optimal operating conditions, the BMS significantly extends battery life, reducing replacement costs.
- Scalability: The system's modular design allows it to be adapted for other Tesla models, including the Model S and Model X.
- Future Prospects: Tesla is investing in innovations like tabless batteries and solid-state technology, which will further enhance the BMS.
-

Case Study 2: Tata Nexon EV - Battery Management System in Indian Conditions

The Tata Nexon EV is one of India's most affordable and popular electric SUVs, designed to perform in diverse and challenging Indian conditions. Its Battery Management System (BMS) ensures reliability and efficiency, making it a practical choice for Indian consumers.

Key Features of the Tata Nexon EV BMS:

1. Battery Monitoring and Safety:
 - The BMS monitors battery parameters like SOC, temperature, and current, providing real-time feedback to the driver.
 - Safety mechanisms are in place to shut down the system in case of overcharging, overheating, or short circuits.

2. Thermal Adaptability:

- Given India's varying climates, the Nexon EV's BMS includes an advanced thermal management system.
- Passive cooling and heat-resistant battery materials are used to handle high temperatures effectively.

3. Range Optimization:

- The BMS optimizes energy usage, delivering an ARAI-certified range of 312 km on a single charge.
- Tata's "Ziptron" technology enhances energy efficiency through regenerative braking and precise power management.

4. Affordability and Localization:

- The BMS is designed with a focus on cost efficiency, leveraging locally sourced components to keep the vehicle affordable.
- This ensures the vehicle remains competitive in India's price-sensitive market.

5. Driver Alerts and Diagnostics:

- The Nexon EV's BMS provides alerts for low battery and excessive temperature.
- The companion mobile app allows users to monitor battery status, schedule charging, and receive diagnostic reports.

Impact and Insights:

- **Affordability and Accessibility:** The Tata Nexon EV democratizes EV adoption in India by offering advanced BMS features at an affordable price point.
- **Adaptability to Indian Conditions:** The system's thermal adaptability ensures reliable performance in hot summers and varying terrains.
- **Consumer Trust:** With its robust safety features and real-time monitoring, the Nexon EV has built consumer trust, becoming a leader in the Indian EV market.
- **Future Prospects:** Tata's investment in expanding charging infrastructure and improving battery chemistry (e.g., lithium iron phosphate) promises further advancements in the Nexon EV's BMS.

Both Tesla and Tata Nexon underscore the pivotal role of a robust Battery Management System (BMS) in ensuring the safety, efficiency, and longevity of electric vehicle (EV) batteries. While Tesla is at the forefront of global EV innovation, leveraging cutting-edge technology and advanced software integration, Tata focuses on delivering cost-effective, region-specific solutions tailored to diverse and challenging markets like India.

Tesla's BMS is built to maximize performance, offering features like precise cell-level monitoring, thermal management with liquid cooling, and over-the-air updates to improve functionality and range. These innovations allow Tesla to achieve exceptional battery ranges, extended lifespans, and high consumer trust in EV adoption. Meanwhile, Tata's BMS balances affordability with reliability, focusing on localized solutions like passive cooling systems for hot climates and regenerative braking optimized for city driving. This approach makes EVs accessible to a price-sensitive demographic while addressing unique operational challenges, such as uneven charging infrastructure and varying weather conditions.

Both manufacturers showcase how BMS innovation is vital for EV adoption. Tesla highlights how advanced technologies can push the boundaries of what EVs can achieve globally, whereas Tata exemplifies how practical, region-specific solutions can accelerate EV adoption in emerging markets. Together, their approaches underline the versatility and critical importance of BMS in shaping the future of the EV industry, addressing safety concerns, performance demands, and market-specific needs.

References:

- [1] <https://www.synopsys.com/glossary/what-is-a-battery-management-system.html>
- [2] https://en.wikipedia.org/wiki/Battery_management_system
- [3] <https://www.sciencedirect.com/topics/engineering/state-of-charge>
- [4] https://en.wikipedia.org/wiki/State_of_charge
- [5] State of Health Prediction of Electric Vehicles' Retired Batteries Based on First-Life Historical Degradation Data Using Predictive Time-Series Algorithms
- [6] Energy Management Systems' Modeling and Optimization in Hybrid Electric Vehicles
- [7] https://www.youtube.com/results?search_query=charge+sensing+in+battery
- [8] <https://www.youtube.com/watch?v=NN03jJmMOB0>
- [9] <https://circuitdigest.com/microcontroller-projects/arm7-lpc2148-adc-tutorial-to-measure-analog-voltage>
- [10] <http://mocha-java.uccs.edu/ECE5720/index.html>

Appendix-A

Source Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// Define the I2C address for the LCD
#define I2C_ADDR 0x27
// Initialize the LCD
LiquidCrystal_I2C lcd(I2C_ADDR, 16, 2);
// Pin definitions
const int voltagePin = A0;
const int currentPin = A1;
const int tempPin = A2;
const int SOCPin = A3;
// Constants for calculations
const float voltageDividerRatio = 5.0; // Adjust based on your voltage sensor module
const float ACS712_sensitivity = 0.066; // Sensitivity for ACS712 30A sensor in V/A
void setup() {
    // Initialize the LCD
    lcd.begin(16, 2);
    lcd.backlight();
    // Initialize serial communication for debugging
    Serial.begin(9600);
    Serial.println("Setup complete.");
}

void loop() {
    // Read sensors and calculate voltage
    float batteryVoltage = readBatteryVoltage();

    // Read the current sensor
    float current_mA = readCurrent();

    // Read the temperature sensor
    float temperature = readTemperature();

    // Estimate SOC based on battery voltage
    float SOC = estimateSOC(batteryVoltage);
```

```

// Display data on LCD and Serial Monitor
displayData(batteryVoltage, SOC, current_mA, temperature);

// Print debug information
Serial.print("Battery Voltage: ");
Serial.print(batteryVoltage);
Serial.print("V, SOC: ");
Serial.print(SOC);
Serial.print("%, Current: ");
Serial.print(current_mA);
Serial.print("mA, Temperature: ");
Serial.print(temperature);
Serial.println("C");

delay(1000); // Adjust delay as needed
}

// Function to read battery voltage
float readBatteryVoltage() {
    int voltageReading = analogRead(voltagePin);
    float voltage = (voltageReading * (5.0 / 1023.0)) * voltageDividerRatio;
    Serial.print("Voltage Reading: ");
    Serial.println(voltageReading);
    Serial.print("Calculated Voltage: ");
    Serial.println(voltage);
    return voltage;
}

// Function to read current
float readCurrent() {
    int currentReading = analogRead(currentPin);
    float current = ((currentReading - 512) * (5.0 / 10230.1)) / ACS712_sensitivity * 100; //
    Convert to mA
    Serial.print("Current Reading: ");
    Serial.println(currentReading);
    Serial.print("Calculated Current: ");
    Serial.println(current);
    return current;
}

// Function to read temperature

```

```

float readTemperature() {
    int tempReading = analogRead(tempPin);
    float temperature = (tempReading * (5.0 / 1023.0)) * 100.0; // for LM35, 10mV/°C
    Serial.print("Temperature Reading: ");
    Serial.println(tempReading);
    Serial.print("Calculated Temperature: ");
    Serial.println(temperature);
    return temperature;
}

// Function to estimate SOC based on battery voltage
float estimateSOC(float voltage) {
    const float voltageMin = 3.0; // Minimum battery voltage (fully discharged)
    const float voltageMax = 4.2; // Maximum battery voltage (fully charged)

    // Clamp voltage to ensure it's within expected range
    if (voltage < voltageMin) {
        voltage = voltageMin;
    } else if (voltage > voltageMax) {
        voltage = voltageMax;
    }

    // Linear mapping formula with floating-point precision
    float soc = ((voltage - voltageMin) / (voltageMax - voltageMin)) * 100.0;
    Serial.print("Voltage for SOC calculation: ");
    Serial.println(voltage);
    Serial.print("Calculated SOC: ");
    Serial.println(soc);

    return soc;
}

// Function to display data on LCD and Serial Monitor
void displayData(float voltage, float soc, float current, float temperature) {
    lcd.clear();

    // Display voltage and SOC on the first line
    lcd.setCursor(0, 0);
    lcd.print("V:");
    lcd.print(voltage, 2);
    lcd.print("V SOC:");

```

```

lcd.print(soc, 1);
lcd.print("%");

// Display current and temperature on the second line
lcd.setCursor(0, 1);
lcd.print("I:");
lcd.print(current, 1);
lcd.print("mA T:");
lcd.print(temperature, 1);
lcd.print("C");

// Print to Serial Monitor for debugging
Serial.print("Voltage: ");
Serial.print(voltage, 2);
Serial.print(" V, ");
Serial.print("SOC: ");
Serial.print(soc, 1);
Serial.print(" %, ");
Serial.print("Current: ");
Serial.print(current, 1);
Serial.print(" mA, ");
Serial.print("Temperature: ");
Serial.print(temperature, 1);
Serial.println(" C");
}

```

First ESP32 Code handles sensing and transmitting data.

```

#include <esp_now.h>
#include <WiFi.h>
#include "DHT.h"
#include <LiquidCrystal_I2C.h>
#define DHTPIN 5
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27,16,2);

```

```

#define ANALOG_IN_PIN 36
#define REF_VOLTAGE 3.3
#define ADC_RESOLUTION 4096.0
#define R1 29000.0
#define R2 7500.0

float tot;

int errorreceived = 0;

uint8_t broadcastAddress[] = {0x30, 0xC6, 0xF7, 0x99, 0x51, 0x18};

typedef struct struct_message
{
    float a;
    float b;
    float c;
    float d;
} struct_message;

typedef struct error_message
{
    int err;
} error_message;

struct_message myData;
error_message incomingReadings;
esp_now_peer_info_t peerInfo;

void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status)
{
    Serial.print("Data Sent to RSU : ");
    lcd.setCursor(0,0);
    lcd.print("Data Sent to RSU");
    lcd.setCursor(0,1);
    lcd.print(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : " Delivery Fail ");
    delay(1500);

```

```

}

void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len)
{
    memcpy(&incomingReadings, incomingData, sizeof(incomingReadings));
    Serial.print("Bytes received: ");
    Serial.println(len);
    errorreceived = incomingReadings.err;
}

void setup()
{
    lcd.init();
    lcd.backlight();
    Serial.begin(115200);
    analogSetAttenuation(ADC_11db);
    dht.begin();
    WiFi.mode(WIFI_STA);
    if (esp_now_init() != ESP_OK)
    {
        Serial.println("Error initializing ESP-NOW");
        return;
    }
    esp_now_register_send_cb(OnDataSent);
    memcpy(peerInfo.peer_addr, broadcastAddress, 6);
    peerInfo.channel = 0;
    peerInfo.encrypt = false;
    if (esp_now_add_peer(&peerInfo) != ESP_OK)
    {
        Serial.println("Failed to add peer");
        return;
    }
}

```

```

    esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv))
    lcd.setCursor(0,0);
    lcd.print(" Cloud Based ");
    lcd.setCursor(0,1);
    lcd.print(" Autonomous ");
    delay(2000);
    lcd.setCursor(0,0);
    lcd.print("Vehicle Tracking");
    lcd.setCursor(0,1);
    lcd.print(" System ");
    delay(2000);
}
void loop()
{
    int adc_value = analogRead(ANALOG_IN_PIN);
    float voltage_adc = ((float)adc_value * REF_VOLTAGE) / ADC_RESOLUTION;
    float voltage_in = voltage_adc * (R1 + R2) / R2;
    for(int i = 1; i<=5; i++)
    {
        // read the input on analog pin 0:
        int sensorValue = analogRead(35);
        // Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
        float voltage = sensorValue * (3.3 / 4095.0);
        float currentv = voltage - 2.25;
        float current = 2222.2222 * currentv;
        tot = tot + current;
        delay(200);
    }
    float avg = tot/5;
    // print out the value you read

```

```

if(avg < 150)
avg = 0;
myData.a = voltage_in;
myData.b = avg / 1000;
myData.c = dht.readTemperature();
myData.d = dht.readHumidity();
lcd.setCursor(0,0);
lcd.print("V:   I:   ");
lcd.setCursor(0,1);
lcd.print("T:   H:   ");
lcd.setCursor(2,0);
lcd.print(voltage_in,2);
lcd.setCursor(10,0);
lcd.print(myData.b,2);
lcd.setCursor(2,1);
lcd.print(myData.c,2);
lcd.setCursor(10,1);
lcd.print(myData.d,2);

Serial.print("Error Code : ");
Serial.println(errorreceived);
delay(2000);
if(errorreceived == 1)
{
  lcd.setCursor(0,0);
  lcd.print("HIGH TEMPERATURE");
  lcd.setCursor(0,1);
  lcd.print("PLEASE COOL DOWN");
  delay(3000);
}

```

```

else if(errorreceived == 2)
{
  lcd.setCursor(0,0);
  lcd.print(" Battery Low ");
  lcd.setCursor(0,1);
  lcd.print(" Please Charge ");
  delay(3000);
}
// Send message via ESP-NOW
esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
if (result == ESP_OK)
{
  Serial.println("Sent with success");
}
else
{
  Serial.println("Error sending the data");
}
tot = 0;
}

```

Second ESP32 Code handles acknowledgment and relaying data.

```

#include <esp_now.h>
#include <WiFi.h>
#include <HardwareSerial.h>
HardwareSerial SerialPort(2);
uint8_t broadcastAddress[] = {0x30, 0xC6, 0xF7, 0x99, 0x63, 0xE0};
// Structure example to receive data
// Must match the sender structure
typedef struct struct_message {

```

```

    float a;

    float b;

    float c;

    float d;
} struct_message;
typedef struct error_message
{
    int err;
} error_message;
// Create a struct_message called myData
struct_message myData;
error_message incomingReadings;
esp_now_peer_info_t peerInfo;
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status)
{
    Serial.print("Data Sent to RSU : ");

    Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery
Fail");
}
// callback function that will be executed when data is received
void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
    memcpy(&myData, incomingData, sizeof(myData));

    Serial.print("Bytes received: ");

    Serial.println(len);

    Serial.print("Battery Voltage : ");

    Serial.println(myData.a);

    Serial.print("Battery Current : ");

    Serial.println(myData.b);

    Serial.print("Temperature    : ");

    Serial.println(myData.c);

```

```
Serial.print("Humidity    : ");
Serial.println(myData.d);
Serial.println();
SerialPort.write('a');
delay(5);
SerialPort.print(myData.a);
delay(5);
SerialPort.write('b');
delay(5);
SerialPort.print(myData.b);
delay(5);
SerialPort.write('d');
delay(5);
SerialPort.print(myData.c);
delay(5);
SerialPort.write('e');
delay(5);
SerialPort.print(myData.d);
delay(5);
SerialPort.write('c');
delay(5);
}

void setup()
{
    // Initialize Serial Monitor
    Serial.begin(115200);
    SerialPort.begin(15200, SERIAL_8N1, 16, 17);
    // Set device as a Wi-Fi Station
    WiFi.mode(WIFI_STA);
    // Init ESP-NOW
```

```

if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
}
esp_now_register_send_cb(OnDataSent);
memcpy(peerInfo.peer_addr, broadcastAddress, 6);
peerInfo.channel = 0;
peerInfo.encrypt = false;
if (esp_now_add_peer(&peerInfo) != ESP_OK)
{
    Serial.println("Failed to add peer");
    return;
}
// Once ESPNow is successfully Init, we will register for recv CB to
// get recv packer info
esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv));
}
void loop()
{
    if(myData.c > 50)
    {
        incomingReadings.err = 1;
    }
    else
    {
        incomingReadings.err = 0;
    }
    if(myData.a < 8)
    {
        incomingReadings.err = 2;
    }
}

```

```

    }
    else
    {
        incomingReadings.err = 0;
    }

    esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &incomingReadings,
    sizeof(incomingReadings));

    delay(500);
}

```

Third ESP32 Code handles cloud communication.

```

#include <WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
#include <HardwareSerial.h>
#define WLAN_SSID "Cloud"
#define WLAN_PASS "12345678"
#define AIO_SERVER "io.adafruit.com"
#define AIO_SERVERPORT 1883 // use 8883 for SSL
#define AIO_USERNAME "SnehaM10"
#define AIO_KEY "aio_euuX15OdOZuLMgQBGqDpTuWDY6tw"

WiFiClient client;

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT,
AIO_USERNAME, AIO_KEY);

Adafruit_MQTT_Publish batvoltage = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Battery Voltage");

Adafruit_MQTT_Publish batcurrent = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Current Consumption");

Adafruit_MQTT_Publish temperature = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Temperature");

Adafruit_MQTT_Publish humidity = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME
"/feeds/Humidity");

HardwareSerial SerialPort(2);

```

```

float batv, bati, temp, hum;
char inChar;
unsigned long previousMillis = 0;
const long interval = 20000;
void MQTT_connect();
void setup()
{
  Serial.begin(115200);
  SerialPort.begin(15200, SERIAL_8N1, 16, 17);
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Wifi Connected");
}
void loop()
{
  MQTT_connect();
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval)
  {
    previousMillis = currentMillis;
    batvoltage.publish(batv);
    batcurrent.publish(bati);
    temperature.publish(temp);
    humidity.publish(hum);
  }
  while (SerialPort.available() > 0)

```

```

{
    inChar = SerialPort.read();
    while (inChar != 'a')
    {
        inChar = SerialPort.read();
    }
    batv = SerialPort.parseFloat();
    inChar = SerialPort.read();

    if (inChar == 'b')
    {
        bati = SerialPort.parseFloat();
    }
    inChar = SerialPort.read();
    if (inChar == 'd')
    {
        temp = SerialPort.parseFloat();
    }
    inChar = SerialPort.read();
    if (inChar == 'e')
    {
        hum = SerialPort.parseFloat();
    }
    inChar = SerialPort.read();
    if (inChar == 'c')
    {
        break;
    }
}
}

```

```
void MQTT_connect() {  
  int8_t ret;  
  // Stop if already connected.  
  if (mqtt.connected()) {return;}  
  Serial.println("Connecting to MQTT..");  
  uint8_t retries = 3;  
  while ((ret = mqtt.connect()) != 0) { // connect will return 0 for connected  
    Serial.println(mqtt.connectErrorString(ret));  
    Serial.println("Retrying MQTT connection in 5 seconds...");  
    mqtt.disconnect();  
    delay(5000); // wait 5 seconds  
    retries--;  
    if (retries == 0) {  
      // basically die and wait for WDT to reset me  
      while (1);  
    }  
  }  
  Serial.println("MQTT Connected!");  
}
```